

# Aufgabe 2: Simultane Labyrinth

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:  
Quirin Klemm

28. September 2025

## Inhaltsverzeichnis

|          |                                  |           |
|----------|----------------------------------|-----------|
| <b>1</b> | <b>Lösungsidee</b>               | <b>2</b>  |
| 1.1      | Erstellung der Matrix . . . . .  | 2         |
| 1.2      | Lösen der Labyrinth . . . . .    | 2         |
| 1.3      | Gruben . . . . .                 | 3         |
| <b>2</b> | <b>Umsetzung</b>                 | <b>4</b>  |
| 2.1      | Erstellung der Matrix . . . . .  | 4         |
| 2.2      | Lösen der Labyrinth . . . . .    | 5         |
| <b>3</b> | <b>Laufzeit und Diskussion</b>   | <b>7</b>  |
| 3.1      | Komplexität der Lösung . . . . . | 7         |
| 3.2      | Laufzeit . . . . .               | 7         |
| 3.3      | Fazit . . . . .                  | 8         |
| <b>4</b> | <b>Beispiele</b>                 | <b>10</b> |
| 4.1      | labyrinth0.txt . . . . .         | 10        |
| 4.2      | labyrinth1.txt . . . . .         | 10        |
| 4.3      | labyrinth2.txt . . . . .         | 11        |
| 4.4      | labyrinth3.txt . . . . .         | 11        |
| 4.5      | labyrinth4.txt . . . . .         | 12        |
| 4.6      | labyrinth5.txt . . . . .         | 12        |
| 4.7      | labyrinth6.txt . . . . .         | 13        |
| 4.8      | labyrinth7.txt . . . . .         | 13        |
| 4.9      | labyrinth8.txt . . . . .         | 14        |
| 4.10     | labyrinth9.txt . . . . .         | 14        |
| <b>5</b> | <b>Quellcode</b>                 | <b>15</b> |
| 5.1      | Move Klasse . . . . .            | 15        |
| 5.2      | Solving Klasse . . . . .         | 15        |
| 5.3      | Cost Klasse . . . . .            | 16        |
| 5.4      | main.py . . . . .                | 17        |

## 1 Lösungsidee

Gesucht ist eine möglichst kurze Abfolge von Schritten, die beide Labyrinth mit den selben Befehlen löst. Dafür wird zunächst für jedes Labyrinth eine Matrix erstellt (Abbildung 1), in der für jede Position ein Tuple aus einem Kostenwert und einer Richtung gespeichert wird. Die gespeicherte Richtung zeigt den optimalen Folgeschritt zum Ziel und ist dargestellt als

| Code | Richtung |
|------|----------|
| 0    | Rechts   |
| 1    | Links    |
| 2    | Oben     |
| 3    | Unten    |

Tabelle 1: Codierung der Richtungen im Labyrinth

Der Kostenwert gibt an, wie viele Schritte bis zum Ziel benötigt werden. Ein Labyrinth mit dazugehöriger Matrix könnte so aussehen:

|      |      |      |     |        |
|------|------|------|-----|--------|
| 13;3 | 14;1 | 5;0  | 4;0 | 3;3    |
| 12;3 | 13;1 | 14;1 | 5;2 | 2;3    |
| 11;3 | 8;0  | 7;0  | 6;2 | 1;3    |
| 10;0 | 9;2  | 8;2  | 7;2 | 0;null |

Abbildung 1: Labyrinth und Matrix

### 1.1 Erstellung der Matrix

Zu Beginn wird eine Matrix erstellt, deren Werte für alle Positionen zunächst leer sind. Ausgehend vom Zielpunkt durchläuft der Algorithmus rekursiv alle erreichbaren Nachbarpositionen. Für jeden Nachbarn wird in der Matrix ein Kostenwert eingetragen, der um eins höher ist als der Kostenwert des aktuellen Punkts, sowie die Richtung, aus der der Nachbar erreicht wurde.

### 1.2 Lösen der Labyrinth

Der Algorithmus funktioniert so, dass er jeweils den momentan besten Weg aus der Liste der bisherigen Wege entnimmt und diesen um einen weiteren Schritt erweitert. Den nächsten Schritt ermittelt er, indem er in der Matrix prüft, welchen Schritt die beiden Labyrinth für ihre jeweilige Position vorschlagen. Dieser Schritt wird an die bisherige Schrittfolge angehängt. Dabei entstehen in der Regel zwei unterschiedliche Wege. Beide werden zur Liste der bisherigen Wege hinzugefügt. Zur Auswertung der bisherigen Wege wird diese Rechnung angewandt:

$$\frac{c_b - c_a}{n} \quad (1)$$

wobei gilt:

- $c_b$  sind die summierten Kosten von beiden Startpositionen

- $c_a$  sind die summierten Kosten von beiden aktuellen Positionen

- $n$  ist die Anzahl der benötigten Schritte

Der Bruch beschreibt also die durchschnittliche Einsparung von Kosten pro Schritt. Je höher der Wert ist, desto effizienter ist der bisherige Pfad. Dabei ist 2 der maximal erreichbare Wert.

Der Weg mit der höchsten Einsparung gilt dabei als momentan bester Weg. Nach jeder Iteration wird

das aktuelle Positionspaar  $((x_1, y_1), (x_2, y_2))$  in einem Dictionary gespeichert, um zu verhindern, dass der Algorithmus das exakt selbe Positionspaar noch einmal besucht. Zusätzlich gibt es auch die Begrenzung  $l$ , die festlegt, wie viele unterschiedliche Wege, sortiert nach ihren Kosten, in der Liste gespeichert werden dürfen. Diese Begrenzung beeinflusst maßgeblich die Laufzeit und Genauigkeit des Algorithmus. In der Liste dürfen die Wege unterschiedliche Werte für  $n$  haben; wichtig ist lediglich, dass ihre Kosten (Gleichung (1)) gering sind. Der gesamte Ablauf wiederholt sich solange, bis beide im Ziel sind.

### 1.3 Gruben

Gruben werden sowohl in der Kostenmatrix als auch beim Lösen wie normale Felder gesehen. Berührt jedoch einer der beiden eine Grube, wird seine Position auf die jeweilige Startposition zurückgesetzt. Dadurch steigen die Gesamtkosten des Wegs so weit an, dass dieser wegen der Begrenzung  $l$  in der Regel aus der Liste der potenziellen Wege entfernt wird.

## 2 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Für beide Labyrinth werden die horizontalen und vertikalen Wände jeweils in separate Listen gespeichert. Auch die Positionen der Gruben werden für jedes Labyrinth in einer eigenen Liste erfasst.

### 2.1 Erstellung der Matrix

Zur Erstellung der Matrix wird die Klasse *cost* benutzt.

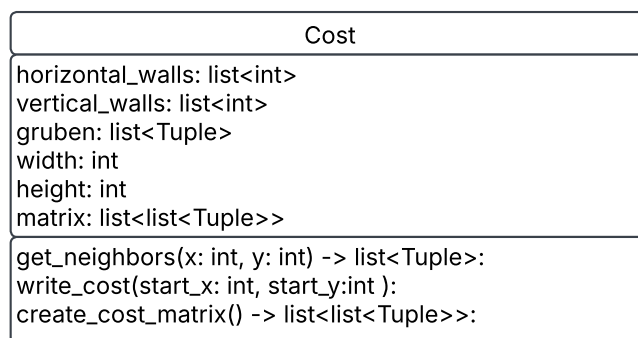


Abbildung 2: Cost Klasse

Für jedes Labyrinth wird ein Cost-Objekt erstellt und die Methode *cost\_obj.create\_cost\_matrix()* aufgerufen. Das Cost-Objekt bekommt als Übergabewerte die horizontalen und vertikalen Wände, die Gruben sowie die Größe des Labyrinths.

```
1 def create_cost_matrix(self) -> list:
```

Listing 1: Methode *create\_cost\_matrix*

Die *create\_cost\_matrix* Methode erstellt zunächst eine leere Liste mit der Dimension [height][width], die als Kostenmatrix dient. Dann ruft sie die Methode *write\_cost* auf, um die Kostenmatrix zu füllen.

```
1 def write_cost(self, start_x: int, start_y: int):
```

Listing 2: Methode *write\_cost*

Zu Beginn der Methode *write\_cost()* wird die Liste *stack* initialisiert. Sie speichert alle Felder, die noch besucht werden müssen. Jeder Eintrag in der Liste besitzt vier Werte: die x-Position, die y-Position, die aktuellen Kosten sowie die entgegengesetzte Richtung, aus welcher das Feld betreten wurde. Die Startposition wird als erstes Element hinzugefügt. Anschließend wird das erste Element der Liste entnommen und in die Kostenmatrix übertragen. Falls es schon in der Kostenmatrix ist, wird es übersprungen. Daraufhin werden alle benachbarten Felder mit der Methode *get\_neighbours* ermittelt und dem *stack* hinzugefügt. Dabei werden die Kosten um eins erhöht und die jeweilige Richtung gespeichert. Das wiederholt sich solange bis der *stack* leer ist.

## 2.2 Lösen der Labyrinth

Zum Lösen der Labyrinth wird die Klasse *Solving* benutzt.

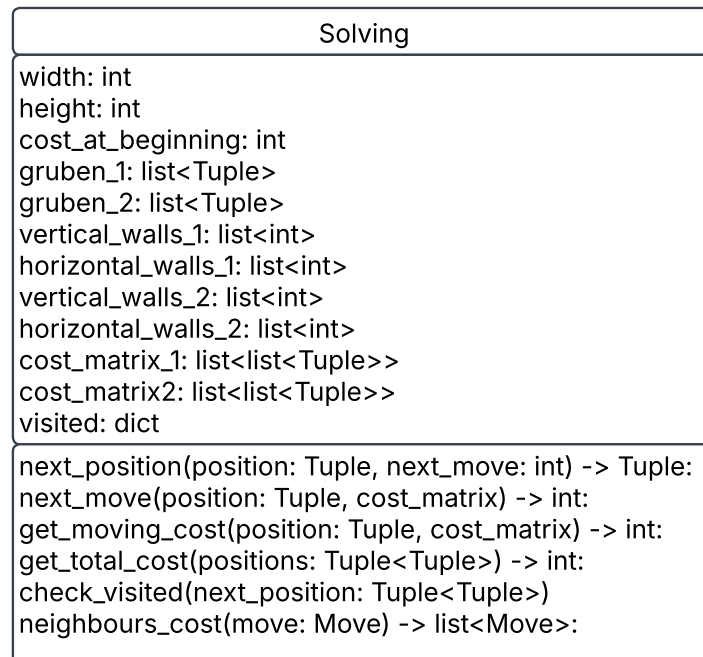


Abbildung 3: Solving Klasse

Die Wege werden mit der Klasse *Move* dargestellt.

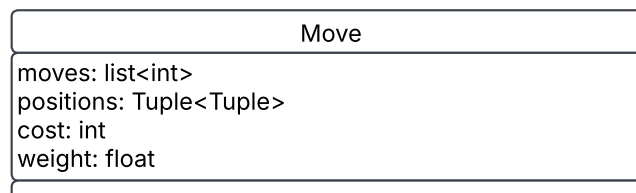


Abbildung 4: Move Klasse

Das Attribut *cost* beschreibt die aufsummierten Kosten der beiden aktuellen Positionen basierend auf den Werten aus den Kostmatrizen. Das Attribut *weight* wird gemäß der Gleichung (1) berechnet.

Zu Beginn wird ein *Solving*-Objekt initialisiert und eine leere Liste *moves* angelegt, in der alle aktuellen Wege gespeichert werden. Anschließend wird ein erstes *Move*-Objekt erzeugt, dessen Startposition  $((0, 0), (0, 0))$  ist und seine *moves* leer sind. In jedem Schritt wird das erste Element aus der *moves* Liste entnommen. Dann wird mit folgendem Aufruf

```
1 new_moves = solving_obj.neighbours_cost(move)
```

eine Liste von möglichen Folgewege erzeugt. Die Liste wird mit den neuen Wegen erweitert, sortiert und auf die *l* besten Wege beschränkt.

```
1 def neighbours_cost(self, move: Move) -> list:
```

Listing 3: Methode *neighbours\_cost*

Zu Beginn der Methode *neighbours\_cost()* werden die beiden Positionen aus dem Attribut *positions* des übergebenen *move*-Objekt genommen. Anschließend werden mit den folgenden Aufrufen

```

1  move_1 = self.next_move(pos1, self.cost_matrix_1)
   move_2 = self.next_move(pos2, self.cost_matrix_2)

```

die jeweils nächsten möglichen Richtungen aus der Kostenmatrix bestimmt, falls das Ziel noch nicht erreicht wurde. Daraufhin werden die neuen zwei Positionspaare berechnet, wenn jeweils in die Richtungen gegangen werden würde.

```

   next_position_1 = self.next_position(move.positions, move_1)
2  next_position_2 = self.next_position(move.positions, move_2)

```

Mit dem folgenden Aufruf wird anschließend überprüft, ob die beiden Positionspaare bereits besucht wurden:

```

   move_1 = self.check_visited(next_position_1, len(move.moves) + 1, move_1)
2  move_2 = self.check_visited(next_position_2, len(move.moves) + 1, move_2)

```

Zum Schluss wird für jeden möglichen Zug ein *Move*-Objekt erstellt und zusammen in einer Liste übergeben. Falls einer der neuen Positionspaare bereits besucht wurde oder bereits im Ziel ist, wird der entsprechende Zug nicht übergeben.

Im folgenden ist die Methode *neighbours\_cost* als Flussdiagramm dargestellt.

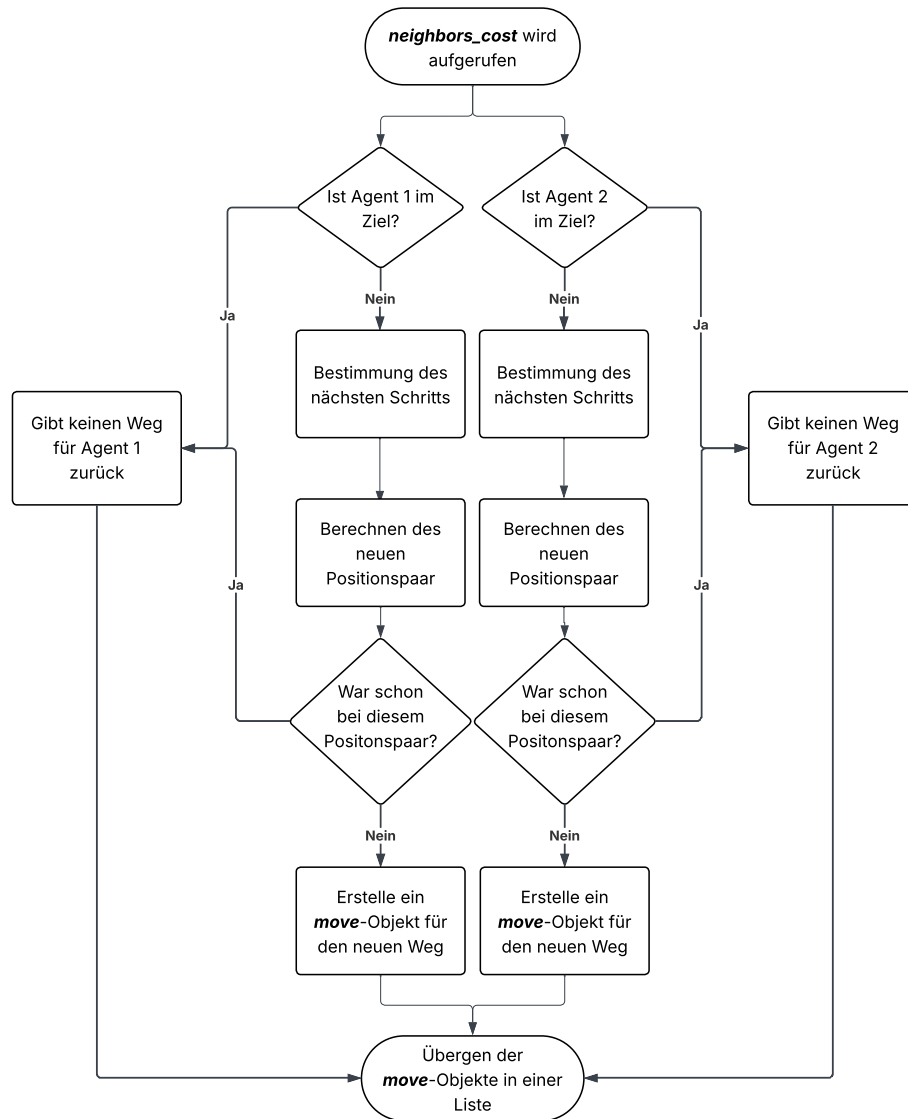


Abbildung 5: *neighbours\_cost* Flussdiagramm

### 3 Laufzeit und Diskussion

#### 3.1 Komplexität der Lösung

Von jeder Position aus existieren im schlimmsten Fall zwei mögliche Schritte, entweder in Richtung des Ziels von Labyrinth 1 oder in Richtung des Ziels von Labyrinth 2. Das führt zu insgesamt  $2^{\text{maximale Anzahl an Schritten}}$  verschiedenen Möglichkeiten. Aufgrund dieser großen Anzahl an Möglichkeiten untersucht der Algorithmus nicht alle Pfade vollständig, sondern macht nur eine Abschätzung. Der hier vorgestellte Ansatz garantiert daher nicht unbedingt den optimalen Lösungsweg, liefert jedoch in akzeptabler Laufzeit eine gute Annäherung. Der Algorithmus ähnelt einer Tiefensuche, da er einige Wege tief verfolgt, während andere nur kurz betrachtet werden.

#### 3.2 Laufzeit

Wie schon oben erwähnt beeinflusst die Begrenzung  $l$ , die Laufzeit und Genauigkeit des Algorithmus stark.

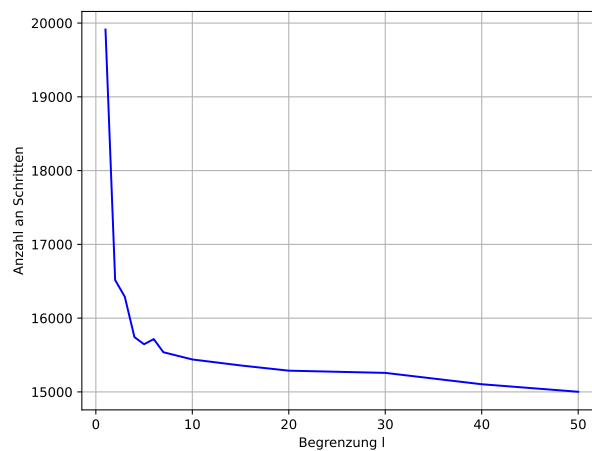


Abbildung 6: Anzahl der Schritte

Abbildung 6 zeigt die Anzahl der Schritte des gefundenen Lösungswegs in Abhängigkeit vom Parameter  $l$ .

Auffällig ist, dass der Graph mit zunehmendem  $l$  gegen die Anzahl der Schritte des optimalen Weges konvergiert. Ebenso erkennbar ist, dass er nicht streng monoton fallend ist. In einem Fall, bei dem die Steigung positiv ist, wird für ein größeren Wert von  $l$  ein schlechterer Pfad gefunden. Dies liegt daran, dass durch den höheren Wert von  $l$  neue Wege betrachtet werden. Diese erscheinen kurzfristig vorteilhafter als die bisherigen Wege und verdrängen dadurch die eigentlich besseren aus der Liste. Das führt insgesamt zu einem schlechteren Weg. Bei größeren  $l$ -Werten tritt dieser Effekt jedoch kaum noch auf, da die Liste groß genug ist, um alle relevanten Pfade zu behalten. Der Einfluss dieser Monotonieveränderungen wird daher mit wachsendem  $l$  vernachlässigbar.

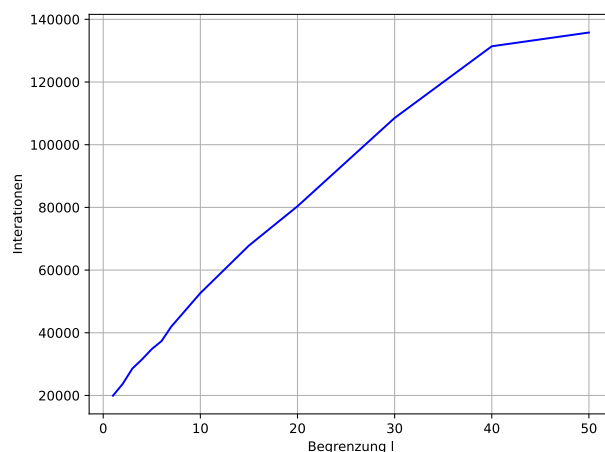


Abbildung 7: maximale Iterationen gegenüber tatsächlichen Iterationen

Abbildung 7 stellt die Anzahl an Iterationen dar, die zur Bestimmung des Lösungswegs benötigt werden in Abhängigkeit vom Parameter  $l$ .

Der Graph steigt mit wachsendem  $l$  stark an.

Allgemein kann die maximale Anzahl an Iterationen in Abhängigkeit von  $l$  wie folgt berechnet werden.

$$l \cdot (2c_1 + c_2) \quad (2)$$

wobei:

- $c_1$  die Kosten vom Startpunkt aus der Kostenmatrix 1 sind
- $c_2$  die Kosten vom Startpunkt aus der Kostenmatrix 2 sind

Der Faktor  $2c_1 + c_2$  beschreibt die maximale Anzahl an Schritten, aus denen ein möglicher Lösungsweg bestehen kann. Zunächst erreicht Agent1 das Ziel mit  $c_1$  Schritten. Anschließend läuft Agent2 denselben Weg zurück in ebenfalls  $c_1$  Schritten und geht dann selber ins Ziel mit  $c_2$  Schritten. Im schlechtesten Fall geht er für jeden Eintrag in der Liste, also  $l$  mal, die maximale Anzahl an Schritten. Daher ergibt sich die Gleichung (2), dies bedeutet, es ergibt sich eine Laufzeit von  $o(l)$  ergibt. Praktisch gesehen ist die Laufzeit aber sehr viel kleiner als die maximale Laufzeit.

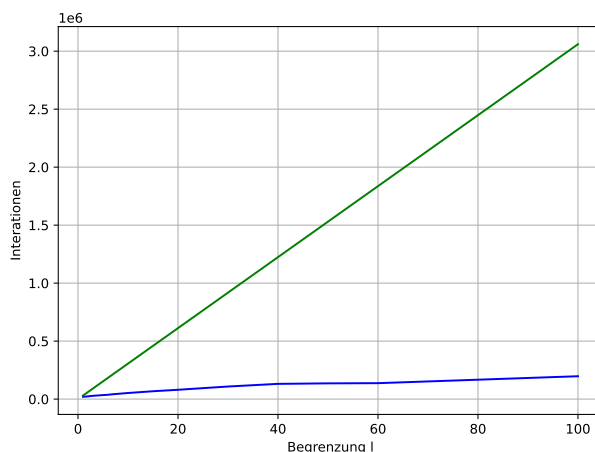


Abbildung 8: maximale Iterationen gegenüber tatsächliche Iterationen

Der grüne Graph zeigt die maximale Anzahl an Iterationen und der blaue die tatsächliche Anzahl an Iterationen.

### 3.3 Fazit

Die Werte von  $l$  stehen in einem linearen Zusammenhang mit der Anzahl der Iterationen, während die Anzahl der Schritte bei größeren  $l$ -Werten gegen das optimale Ergebnis konvergieren. Je größer der Wert

von  $l$ , desto mehr Iteration werden durchgeführt - und desto besser wird das Ergebnis. Die Genauigkeit lässt sich anhand des Parameters  $l$  jedoch nicht allgemein bestimmen, da sie stark von der Struktur des jeweiligen Labyrinths abhängt.

## 4 Beispiele

Zu jedem Beispiel werden der optimale Lösungsweg sowie die Anzahl der Iterationen in Abhängigkeit vom Parameter  $l$  dargestellt. Zusätzlich wird der jeweils beste gefundene Pfad sowie dessen Länge angegeben. Alle Schritte des Pfads werden wie in der Tabelle 1 codiert. Pfade über 1400 Schritte sind gekürzt und nur der Anfang und des Ende dargestellt.

### 4.1 labyrinth0.txt

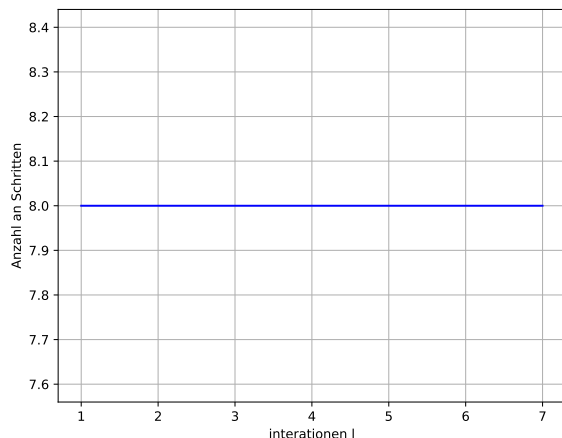


Abbildung 9: Genauigkeit labyrinth0.txt

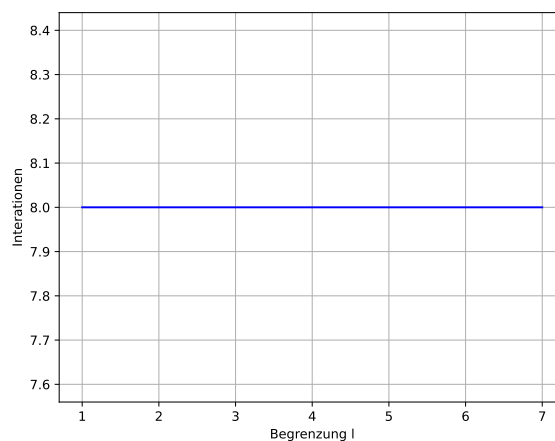


Abbildung 10: Iterationen labyrinth0.txt

Länge des Wegs: 8

Weg: [3, 3, 0, 2, 2, 0, 3, 3]

### 4.2 labyrinth1.txt



Abbildung 11: Genauigkeit labyrinth1.txt

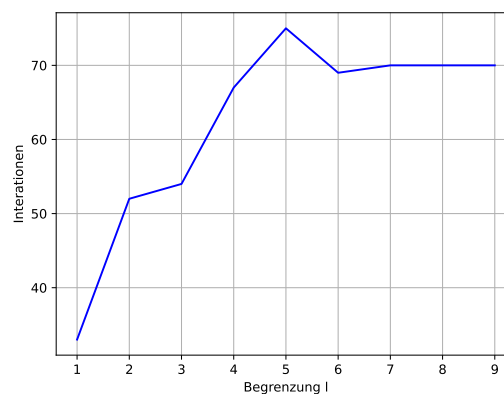


Abbildung 12: Iterationen labyrinth1.txt

Länge des Wegs: 31

Weg: [3, 3, 3, 0, 2, 0, 3, 0, 2, 2, 1, 1, 2, 0, 0, 0, 3, 3, 1, 2, 1, 3, 1, 2, 1, 3, 3, 0, 0, 0, 0]

### 4.3 labyrinth2.txt

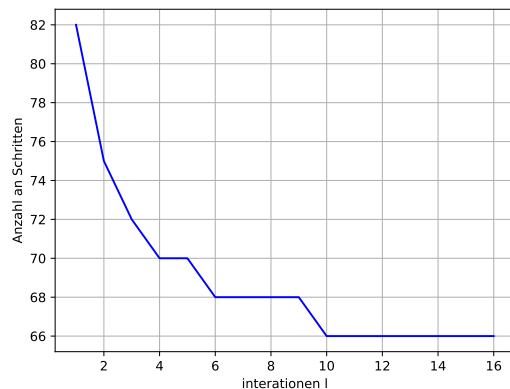


Abbildung 13: Genauigkeit labyrinth2.txt

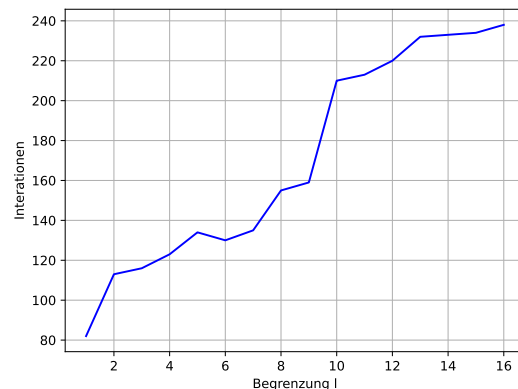


Abbildung 14: Iterationen labyrinth2.txt

Länge des Wegs: 66

Weg: [0, 0, 3, 3, 3, 0, 0, 3, 3, 0, 2, 2, 0, 0, 0, 3, 1, 3, 0, 3, 3, 1, 3, 1, 2, 1, 1, 1, 1, 2, 1, 1, 3, 3, 1, 3, 0, 3, 0, 0, 0, 3, 3, 0, 0, 2, 2, 2, 0, 2, 2, 0, 0, 0, 0, 3, 1, 3, 1, 3, 3, 0, 0, 3]

### 4.4 labyrinth3.txt

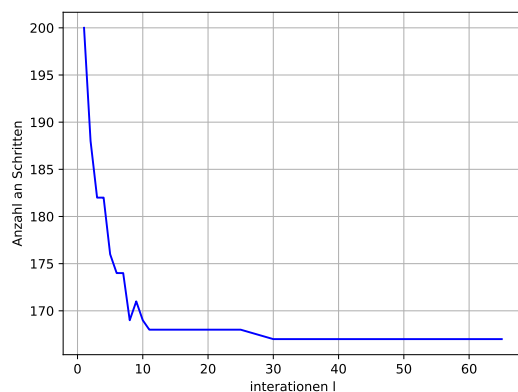


Abbildung 15: Genauigkeit labyrinth3.txt

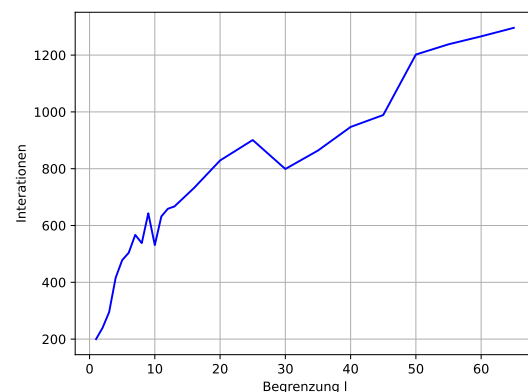


Abbildung 16: Iterationen labyrinth3.txt

Länge des Wegs: 166

Weg: [3, 3, 3, 3, 0, 3, 3, 0, 2, 0, 0, 0, 3, 3, 3, 3, 0, 0, 0, 0, 3, 1, 3, 1, 1, 1, 3, 3, 1, 3, 3, 3, 1, 1, 1, 3, 1, 3, 0, 3, 3, 3, 0, 2, 0, 0, 3, 1, 3, 3, 3, 3, 1, 1, 3, 2, 1, 3, 0, 3, 1, 1, 3, 1, 2, 1, 3, 3, 0, 0, 0, 0, 3, 1, 3, 3, 0, 0, 0, 0, 3, 3, 0, 0, 3, 0, 0, 3, 0, 3, 3, 1, 1, 3, 3, 1, 3, 1, 3, 3, 0, 2, 0, 3, 3, 3, 1, 1, 3, 1, 2, 1, 2, 1, 2, 1, 3, 1, 3, 0, 0, 3, 3, 3, 0, 0, 2, 0, 0, 2, 0, 2, 0, 3, 3, 3, 1, 3, 1, 1, 1, 2, 2, 1, 3, 3, 0, 3, 3, 1, 3, 3, 3, 0, 2, 0, 3, 0, 3, 0, 3, 0, 0, 0, 0, 0]

#### 4.5 labyrinth4.txt

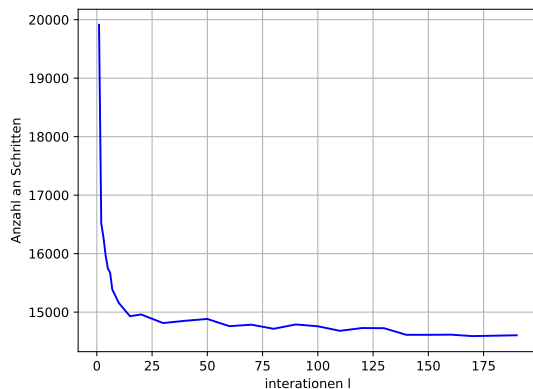


Abbildung 17: Genauigkeit labyrinth4.txt

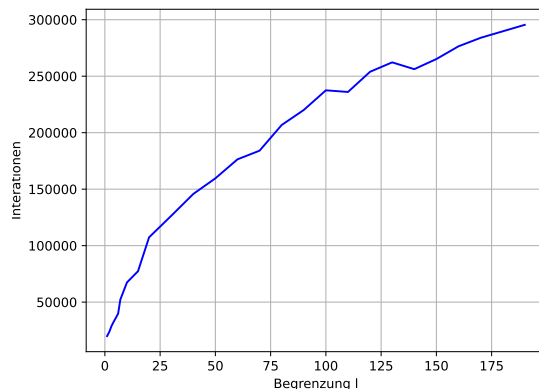


Abbildung 18: Iterationen labyrinth4.txt

Länge des Wegs: 14497

Weg: [0, 0, 3, 0, 2, 0, 0, 0, 3, 1, .... 2, 2, 1, 3, 3, 1, 1, 3, 0, 0, 0, 0, 0] (gekürzt)

#### 4.6 labyrinth5.txt

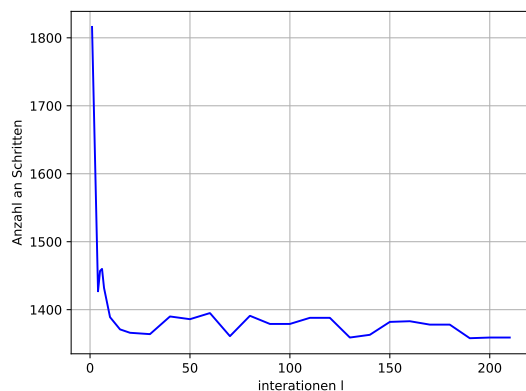


Abbildung 19: Genauigkeit labyrinth5.txt

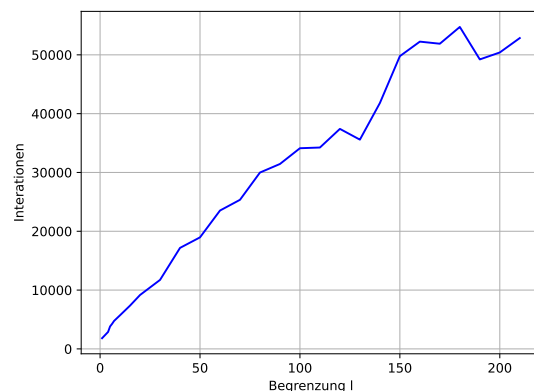


Abbildung 20: Iterationen labyrinth5.txt

Länge des Wegs: 1345

Weg: [0, 0, 3, 3, 0, 0, 2, 0, 2, 0, 0, 3, 1, 1, 3, 3, 1, 1, 3, 0, 3, 0, 3, 0, 0, 1, 3, 0, 3, 1, 1, 1, 3, 0, 3, 1, 3, 0, 3, 3, 1, 3, 3, 0, 0, 3, 3, 0, 0, 3, 3, 0, 3, 3, 0, 0, 2, 0, 3, 0, 3, 3, 1, 3, 0, 0, 0, 0, 3, 0, 3, 1, 3, 3, 3, 1, 1, 3, 3, 3, 3, 0, 0, 0, 2, 2, 0, 2, 2, 0, 0, 2, 0, 0, 0, 2, 0, 0, 2, 0, 3, 0, 2, 0, 0, 0, 3, 3, 3, 3, 0, 3, 1, 3, 1, 1, 3, 3, 3, 3, 0, 0, 3, 0, 3, 3, 3, 3, 1, 1, 1, 1, 3, 1, 3, 0, 3, 1, 1, 3, 3, 0, 0, 2, 2, 0, 2, 0, 2, 2, 0, 3, 3, 3, 0, 2, 2, 2, 0, 0, 3, 0, 0, 3, 0, 2, 0, 3, 3, 3, 3, 1, 3, 1, 1, 3, 3, 0, 3, 0, 3, 0, 0, 3, 0, 3, 0, 0, 3, 0, 2, 2, 0, 0, 0, 0, 3, 0, 2, 2, 0, 0, 0, 3, 0, 0, 0, 2, 2, 1, 1, 2, 0, 0, 2, 0, 3, 0, 0, 0, 2, 1, 1, 1, 2, 2, 2, 2, 1, 2, 2, 0, 3, 0, 0, 0, 2, 2, 1, 1, 1, 1, 2, 2, 0, 2, 0, 2, 2, 0, 0, 2, 2, 2, 2, 0, 3, 3, 0, 0, 2, 0, 2, 2, 0, 3, 0, 2, 0, 3, 3, 0, 0, 0, 3, 0, 3, 3, 1, 3, 3, 1, 1, 3, 1, 3, 3, 0, 3, 3, 1, 1, 3, 0, 3, 0, 2, 0, 3, 0, 0, 3, 3, 1, 3, 3, 1, 1, 3, 0, 0, 3, 1, 3, 0, 3, 3, 0, 2, 2, 0, 2, 2, 0, 3, 0, 0, 0, 2, 0, 2, 2, 1, 2, 2, 0, 0, 2, 0, 0, 2, 0, 3, 0, 3, 1, 3, 3, 3, 0, 0, 2, 0, 2, 2, 2, 2, 0, 0, 3, 3, 0, 3, 0, 0, 0, 3, 3, 2, 0, 0, 0, 3, 3, 0, 2, 2, 2, 1, 2, 1, 1, 2, 2, 0, 2, 0, 3, 3, 3, 0, 0, 0, 0, 3, 0, 2, 0, 2, 2, 2, 2, 0, 0, 3, 0, 3, 0, 3, 3, 3, 3, 3, 3, 2, 2, 0, 0, 0, 0, 0, 3, 3, 3, 0, 3, 1, 1, 1, 1, 3, 1, 1, 3, 1, 2, 1, 2, 1, 1, 3, 1, 1, 3, 0, 3, 3, 1, 1, 3, 1, 3, 0, 3, 1, 1, 3, 3, 3, 3, 3, 3, 1, 1, 3, 0, 3, 3, 3, 1, 3, 1, 1, 1, 3, 1, 1, 2, 1, 3, 1, 1, 1, 1, 1, 2, 1, 3, 0, 3, 1, 3, 1, 1, 1, 3, 1, 1, 3, 3, 3, 3, 3, 0, 3, 1, 3, 3, 1, 3, 3, 0, 0, 3, 0, 3, 3, 1, 1, 3, 1, 3, 0, 2, 2, 0, 3, 0, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 1, 2, 1, 3, 1, 2, 1, 3, 1, 2, 1, 3, 1, 3, 3, 3, 3, 3, 0, 2, 0, 0, 0, 0, 3, 0, 0, 0, 2, 0, 3, 0, 0, 3, 1, 1, 3, 0, 0,

2, 0, 3, 3, 3, 0, 0, 0, 3, 0, 3, 0, 3, 3, 0, 0, 2, 0, 0, 0, 2, 2, 1, 2, 2, 1, 2, 0, 0, 0, 0, 0, 0, 2, 1, 2, 0, 2, 2, 0, 0,  
0, 2, 0, 0, 2, 0, 0, 3, 0, 3, 3, 0, 2, 0, 3, 0, 3, 0, 2, 0, 2, 1, 2, 2, 0, 3, 1, 3, 0, 0, 2, 0, 0, 3, 3, 3, 3, 0, 3, 1, 3,  
0, 0, 0, 3, 3, 1, 2, 3, 1, 2, 0, 2, 0, 0, 1, 3, 0, 3, 3, 3, 3, 1, 3, 0, 0, 0, 3, 3, 0, 3, 3, 0, 3, 0, 3, 3, 1, 2, 1, 1, 3,  
1, 1, 3, 1, 2, 1, 2, 2, 1, 1, 1, 1, 3, 3, 3, 0, 2, 0, 3, 0, 3, 1, 3, 1, 1, 3, 1, 1, 1, 1, 3, 0, 2, 0, 3, 1, 1, 3, 1, 1, 3,  
0, 3, 3, 3, 0, 2, 0, 0, 3, 0, 0, 0, 3, 0, 0, 0, 2, 2, 0, 2, 2, 2, 0, 0, 0, 3, 0, 3, 0, 2, 2, 1, 2, 2, 1, 1, 2, 0, 2, 0, 0,  
0, 2, 0, 3, 0, 2, 0, 2, 0, 3, 3, 0, 0, 2, 0, 2, 2, 0, 0, 0, 3, 0, 0, 0, 2, 0, 3, 0, 3, 0, 3, 3, 3, 0, 2, 0, 2, 2, 1, 2, 0,  
2, 0, 0, 2, 2, 1, 1, 2, 2, 1, 1, 2, 1, 1, 2, 2, 0, 0, 0, 3, 0, 2, 0, 2, 2, 0, 3, 3, 0, 0, 0, 0, 0, 2, 1, 2, 0, 0, 3, 3, 3,  
0, 3, 3, 1, 1, 2, 1, 3, 0, 3, 1, 3, 0, 3, 0, 3, 1, 3, 3, 1, 3, 0, 3, 0, 3, 0, 0, 3, 3, 0, 3, 0, 2, 0, 0, 3, 1, 3, 3, 3, 3,  
0, 3, 1, 1, 3, 1, 3, 1, 3, 1, 2, 2, 1, 1, 3, 1, 1, 1, 1, 3, 1, 1, 2, 0, 2, 0, 0, 2, 2, 1, 2, 0, 0, 3, 0, 0, 2, 0, 0, 1, 2,  
1, 2, 1, 1, 1, 3, 0, 0, 0, 0, 2, 0, 0, 2, 0, 0, 2, 2, 0, 3, 0, 2, 0, 0, 0, 3, 0, 0, 2, 0, 0, 2, 1, 1, 1, 1, 2, 1, 2, 1, 2,  
0, 0, 3, 3, 1, 3, 3, 1, 3, 1, 3, 0, 0, 0, 3, 3, 3, 3, 3, 0, 2, 0, 3, 0, 3, 3, 1, 1, 3, 1, 3, 1, 3, 1, 1, 1, 2, 1, 1, 3, 0,  
2, 0, 3, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 3, 0, 0, 3, 3, 3, 1, 1, 3, 3, 0, 3, 0, 0, 3, 0, 3, 3, 0, 0, 0, 3, 1, 1,  
3, 1, 3, 1, 3, 1, 3, 1, 1, 3, 0, 3, 0, 3, 3, 1, 1, 3, 0, 0, 3, 0, 3, 3, 3, 1, 1, 1, 3, 1, 1, 3, 0, 3, 1, 1, 1, 3, 3, 0, 3,  
1, 2, 1, 3, 3, 3, 1, 1, 2, 2, 1, 1, 3, 3, 0, 3, 3, 0, 3, 0, 3, 3, 3, 1, 3, 3, 0, 0, 2, 0, 3, 3, 0, 3, 3, 0, 2, 0, 2, 2, 0,  
0, 3, 0, 2, 0, 0, 3, 0, 0, 3, 3, 1, 3, 1, 3, 0, 0, 0, 0, 2, 0, 0, 3, 0, 3, 0, 2, 0, 0, 0, 0, 0, 2, 2, 2, 1, 2, 2, 1, 1, 1,  
2, 0, 2, 2, 0, 0, 2, 2, 2, 0, 2, 1, 2, 0, 0, 2, 0, 2, 0, 3, 0, 0, 3, 3, 0, 2, 2, 0, 2, 1, 2, 2, 0, 2, 2, 1, 2, 2, 0, 3, 0,  
0, 0, 2, 0, 2, 0, 3, 0, 2, 0, 2, 2, 2, 0, 0, 0, 0, 3, 3, 3, 3, 0, 0, 3, 0, 3, 0, 0, 3, 3, 0, 0, 3, 3, 0, 3, 0, 0, 3,  
3, 0, 3, 1, 2, 1, 1, 1, 1, 3, 3, 3, 3, 1, 1, 2, 1, 3, 3, 1, 3, 3, 0, 3, 1, 3, 3, 3, 3, 1, 3, 0, 2, 2, 0, 3, 0, 0, 0, 0,  
3, 0, 0, 3, 3, 0, 3, 1, 1, 3, 0, 3, 0, 0, 0, 0, 3, 3, 1, 3, 0, 3, 3, 0, 0, 0, 0, 3, 3, 0, 3, 3, 0, 0, 3]

#### 4.7 labyrinth6.txt

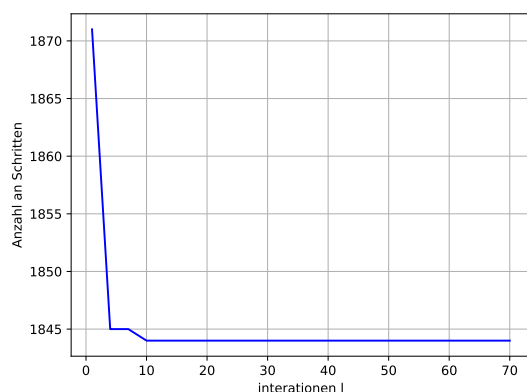


Abbildung 21: Genauigkeit labyrinth6.txt

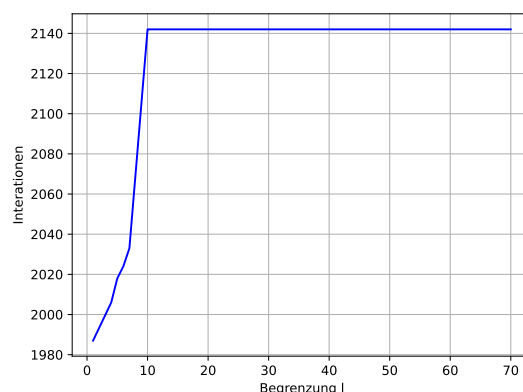


Abbildung 22: Iterationen labyrinth6.txt

Länge des Wegs: 1844

Weg: [0, 0, 0, 3, 3, 3, 3, 3, ..., 0, 0, 3, 0, 0, 0, 2, 2, 0, 0, 0, 3, 3] (gekürzt)

#### 4.8 labyrinth7.txt

Das zweite Labyrinth lässt sich nicht lösen, da eine Grube sich auf der Position (18, 6) befindet. Daher gibt es keinen Weg von Startposition zum Ziel. Der Algorithmus erkennt solche Fälle und gibt aus, dass er keinen Weg gefunden hat.

## 4.9 labyrinth8.txt

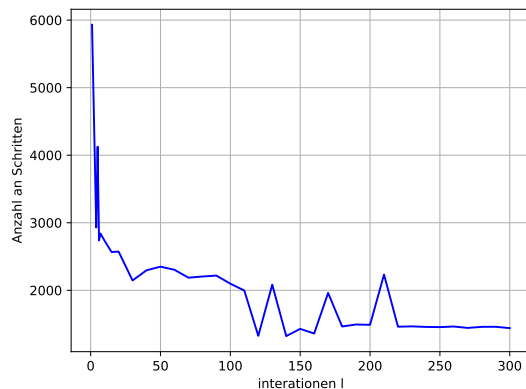


Abbildung 23: Genauigkeit labyrinth8.txt

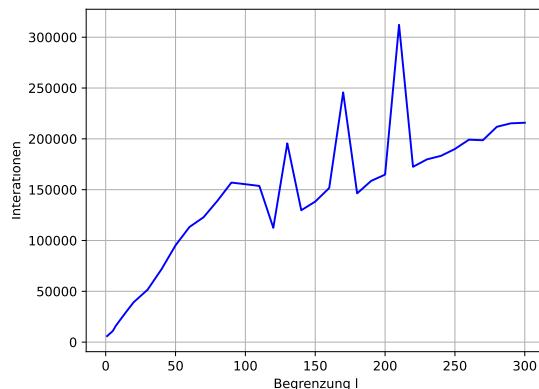


Abbildung 24: Iterationen labyrinth8.txt

Länge des Wegs: 1433

Weg:[3, 0, 0, 3, 3, 3, 3, 0, 3, 3, ..., 0, 3, 3, 3, 0, 2, 0, 3, 3, 0] (gekürzt)

## 4.10 labyrinth9.txt

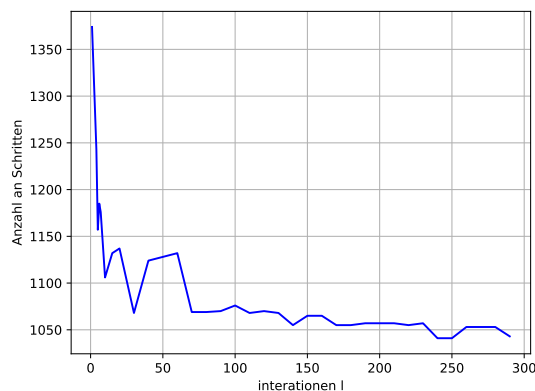


Abbildung 25: Genauigkeit labyrinth9.txt

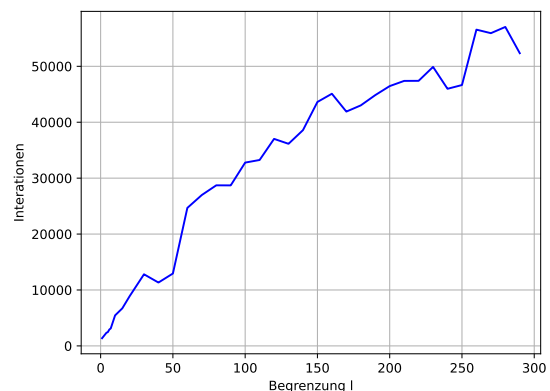


Abbildung 26: Iterationen labyrinth9.txt

Länge des Wegs: 1036

Weg:[3, 0, 3, 0, 0, 3, 0, 0, 2, 0, 0, 3, 3, 3, 0, 2, 0, 2, 0, 0, 3, 3, 3, 3, 0, 0, 2, 2, 0, 0, 3, 0, 3, 0, 3, 3, 0, 3, 1, 3, 1, 3, 0, 3, 3, 0, 3, 0, 2, 0, 0, 2, 0, 0, 2, 2, 0, 2, 0, 3, 3, 0, 3, 3, 3, 0, 0, 0, 0, 3, 3, 0, 0, 2, 0, 0, 3, 3, 3, 3, 0, 0, 3, 1, 3, 1, 1, 1, 3, 0, 0, 3, 0, 3, 1, 3, 1, 3, 1, 3, 0, 0, 3, 3, 0, 3, 3, 0, 3, 1, 3, 3, 1, 1, 2, 2, 1, 1, 3, 3, 1, 3, 3, 3, 3, 0, 3, 3, 0, 0, 3, 3, 0, 2, 0, 0, 0, 0, 3, 1, 3, 0, 0, 0, 0, 3, 3, 1, 3, 0, 0, 2, 0, 0, 3, 3, 0, 3, 0, 3, 3, 1, 1, 3, 1, 3, 1, 3, 3, 1, 1, 2, 1, 3, 3, 1, 3, 3, 0, 0, 3, 0, 0, 0, 0, 3, 0, 2, 0, 3, 1, 1, 3, 1, 3, 0, 3, 0, 3, 0, 0, 3, 0, 2, 0, 0, 0, 2, 0, 3, 3, 3, 0, 0, 3, 0, 0, 0, 2, 0, 2, 0, 0, 2, 0, 2, 1, 2, 2, 1, 2, 0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 0, 0, 3, 1, 3, 0, 3, 1, 1, 3, 0, 0, 0, 3, 0, 3, 0, 0, 3, 3, 0, 3, 3, 0, 3, 1, 1, 1, 3, 1, 3, 1, 3, 3, 3, 3, 0, 2, 2, 0, 2, 2, 0, 2, 0, 0, 2, 0, 0, 0, 3, 3, 1, 1, 2, 1, 1, 3, 0, 0, 3, 0, 3, 3, 3, 3, 3, 1, 3, 3, 1, 1, 3, 1, 3, 0, 0, 3, 0, 3, 0, 3, 1, 3, 0, 2, 2, 2, 0, 0, 0, 3, 3, 1, 3, 1, 1, 3, 1, 3, 0, 3, 1, 3, 1, 1, 1, 3, 0, 3, 3, 3, 0, 0, 2, 0, 2, 0, 0, 3, 3, 3, 3, 0, 3, 0, 3, 0, 0, 2, 1, 2, 0, 2, 0, 0, 2, 0, 0, 2, 1, 2, 1, 1, 2, 0, 0, 2, 2, 0, 3, 0, 3, 3, 0, 0, 2, 2, 0, 3, 3, 0, 3, 3, 0, 0, 3, 3, 0, 0, 0, 3, 3, 3, 3, 3, 1, 1, 3, 1, 2, 2, 2, 1, 1, 3, 0, 3, 0, 0, 3, 0, 2, 0, 3, 0, 3, 3, 1, 1, 1, 1, 3, 0, 0, 3, 0, 3, 1, 3, 3, 0, 3, 1, 3, 1, 2, 2, 1, 3, 3, 0, 3, 1, 1, 3, 1, 3, 0, 3, 3, 0, 0, 3, 0, 0, 0, 2, 2, 2, 0, 0, 3, 0, 0, 0, 2, 2, 1,

```

2, 1, 2, 0, 0, 0, 3, 0, 3, 0, 0, 3, 3, 3, 0, 2, 2, 2, 0, 0, 2, 2, 2, 1, 2, 2, 2, 1, 2, 0, 3, 0, 3, 0, 2, 0, 0, 0, 2, 0, 3,
3, 3, 0, 3, 0, 3, 1, 1, 3, 3, 0, 3, 0, 2, 0, 3, 0, 2, 0, 2, 2, 2, 2, 1, 2, 1, 2, 0, 2, 0, 0, 2, 0, 3, 0, 0, 3, 3, 3, 3,
1, 3, 0, 3, 1, 3, 0, 0, 0, 3, 1, 3, 3, 3, 1, 3, 3, 3, 1, 3, 1, 1, 2, 2, 1, 1, 1, 1, 2, 1, 3, 1, 3, 3, 1, 1, 3, 0, 3, 0,
3, 0, 0, 0, 3, 1, 3, 3, 0, 3, 0, 2, 0, 0, 3, 0, 3, 3, 3, 3, 3, 3, 1, 3, 1, 3, 1, 3, 1, 3, 0, 3, 0, 0, 0, 2, 0, 3, 0, 2,
0, 3, 0, 3, 3, 0, 2, 0, 2, 2, 0, 2, 0, 0, 3, 0, 2, 0, 3, 3, 1, 1, 3, 3, 0, 3, 0, 2, 0, 3, 0, 0, 3, 0, 3, 3, 3, 0, 2, 0,
3, 3, 3, 3, 1, 3, 0, 3, 1, 3, 0, 0, 2, 0, 2, 2, 1, 1, 2, 1, 2, 2, 0, 0, 3, 0, 3, 3, 3, 2, 0, 3, 0, 3, 0, 3, 0, 0, 0, 3, 3,
1, 1, 3, 3, 1, 3, 0, 3, 0, 3, 1, 3, 0, 3, 1, 3, 3, 3, 1, 1, 3, 3, 1, 2, 1, 2, 2, 1, 2, 2, 2, 1, 3, 1, 1, 1, 3, 3, 3, 0, 3,
3, 3, 1, 3, 3, 3, 0, 0, 3, 1, 3, 0, 2, 0, 0, 0, 0, 0, 2, 0, 0, 3, 3, 3, 3, 0, 3, 0, 3, 3, 0, 3, 0, 0, 3, 3, 0, 3, 1, 3, 3,
0, 0, 0, 0, 0, 3, 3, 0, 2, 0, 2, 2, 0, 3, 0, 0, 2, 1, 2, 0, 3, 0, 3, 0, 2, 0, 0, 0, 0, 2, 2, 2, 1, 2, 2, 0, 2, 0, 3, 0, 3,
0, 3, 0, 2, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 0, 3, 0, 0, 0, 0, 0, 2, 2, 0, 2, 0, 3, 3, 0, 3, 3, 3, 3, 0, 3, 0, 2, 0, 3, 0, 0,
3, 0, 2, 0, 0, 0, 3, 0, 0, 0, 3, 1, 3, 1, 1, 3, 3, 3, 0, 3, 0, 3, 3, 0, 3, 1, 3, 0, 0, 3, 0, 2, 2, 0, 0, 3, 0, 3, 0, 3, 0,
0, 2, 2, 2, 0, 3, 3, 0, 0, 3, 3, 3, 3]

```

## 5 Quellcode

### 5.1 Move Klasse

```

class Move:
2
    def __init__(self, moves, positions, cost, heights_cost, weight=None):
4
        self.moves = moves
        self.positions = positions
6
        self.cost = cost
        self.weight = weight if weight is not None else (heights_cost - cost) / len(moves) # moves are sort

```

### 5.2 Solving Klasse

```

class Solving:
2
    def __init__(self, cost_matrix_1, cost_matrix_2, vertical_walls_1, horizontal_walls_1,
        vertical_walls_2, horizontal_walls_2, gruben_1, gruben_2, width, height):
4
        self.width = width
        self.height = height
6
        self.cost_at_beginning = cost_matrix_1[0][0][0] + cost_matrix_2[0][0][0]

8
        self.gruben_1 = gruben_1
        self.gruben_2 = gruben_2

10
        self.vertical_walls_1 = vertical_walls_1
12
        self.horizontal_walls_1 = horizontal_walls_1

14
        self.vertical_walls_2 = vertical_walls_2
        self.horizontal_walls_2 = horizontal_walls_2

16
        self.cost_matrix_1 = cost_matrix_1
18
        self.cost_matrix_2 = cost_matrix_2

20
        self.visited = {}
        # Funktionen und Bewegungsdeltas nur einmal definieren
22
        self.move_funcs = [test_right, test_left, test_up, test_down]
        self.move_deltas = [(1, 0), (-1, 0), (0, -1), (0, 1)]

24
    def next_position(self, position, next_move):
26
        mf = self.move_funcs
        md = self.move_deltas
28
        pos1 = position[0]
        pos2 = position[1]

30
        # Für die erste Position
32
        if pos1 != (self.width - 1, self.height - 1):
            matrix1 = self.vertical_walls_1 if next_move < 2 else self.horizontal_walls_1
34
            if mf[next_move](pos1[0], pos1[1], matrix1):
                pos1 = (pos1[0] + md[next_move][0], pos1[1] + md[next_move][1])
36
            if pos1 in self.gruben_1:
                pos1 = (0, 0)
38

```

```

# Für die zweite Position
40 if pos2 != (self.width - 1, self.height - 1):
    matrix2 = self.vertical_walls_2 if next_move < 2 else self.horizontal_walls_2
42     if mf[next_move](pos2[0], pos2[1], matrix2):
        pos2 = (pos2[0] + md[next_move][0], pos2[1] + md[next_move][1])
44     if pos2 in self.gruben_2:
        pos2 = (0, 0)
46 return pos1, pos2

48 def next_move(self, pos, cost_matrix):
    w = self.width
    h = self.height
    return None if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos[0]][1]
52
54 def get_moving_cost(self, pos, cost_matrix):
    w = self.width
    h = self.height
    return 0 if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos[0]][0]
56
58 def get_total_cost(self, positions):
    pos1, pos2 = positions
    return self.get_moving_cost(pos1, self.cost_matrix_1) + self.get_moving_cost(pos2, self.cost_matrix_2)
60
62 def check_visited(self, next_position, length, move):
    # Erstelle einen Schlüssel als Tupel, das beide Positionen enthält
64     key = (next_position[0][0], next_position[0][1],
            next_position[1][0], next_position[1][1])
66     if key in self.visited and self.visited[key] <= length:
        return None
68     self.visited[key] = length
    return move
70
72 def neighbours_cost(self, move: Move):
    # Berechne beide Positionen anhand der Bewegungsfolge
    pos1 = move.positions[0]
    pos2 = move.positions[1]
74
76     # Ermittle den nächsten Zug für beide Positionen
    move_1 = self.next_move(pos1, self.cost_matrix_1)
    move_2 = self.next_move(pos2, self.cost_matrix_2)
78
80     # Nächsten Positionspaare berechnen
    next_position_1 = self.next_position(move.positions, move_1) if move_1 is not None else None
    next_position_2 = self.next_position(move.positions, move_2) if move_2 is not None else None
82
84     # Testen ob die Positionspaare schon einmal da waren
    move_1 = self.check_visited(next_position_1, len(move.moves) + 1, move_1) if move_1 is not None else None
    move_2 = self.check_visited(next_position_2, len(move.moves) + 1, move_2) if move_2 is not None else None
86
88     results = []
    if move_1 is not None:
        results.append(Move(move.moves + [move_1], next_position_1, self.get_total_cost(next_position_1),
90                                self.cost_at_beginning))
    if move_2 is not None:
        results.append(Move(move.moves + [move_2], next_position_2, self.get_total_cost(next_position_2),
92                                self.cost_at_beginning))
94     return results
96

```

### 5.3 Cost Klasse

```

1 class Cost:
2
3     def write_cost(self, start_x, start_y):
        stack = [(start_x, start_y, 0, None)]
4
5         while stack:
6             x, y, cost, direction = stack.pop()
7             if self.matrix[y][x] != 0:
8                 continue
9

```

```

11         self.matrix[y][x] = (cost, direction)

13
14         a = self.get_neighbors(x, y)
15         for neighbor in a:
16             nx, ny, ndirection = neighbor
17             if self.matrix[ny][nx] == 0:
18                 stack.append((nx, ny, cost + 1, ndirection))
19
20
21     def create_cost_matrix(self):
22         self.matrix = [[0 for _ in range(self.width)] for _ in range(self.height)]
23         self.write_cost(self.width - 1, self.height - 1)
24         return self.matrix
25

```

## 5.4 main.py

```

def create_matrix(data, height):
2     vertical_walls = [data.pop(0) for _ in range(height)]
3     horizontal_walls = [data.pop(0) for _ in range(height - 1)]
4     gruben_anzahl = int(data.pop(0)[0])
5     gruben = [tuple(data.pop(0)) for _ in range(gruben_anzahl)]
6     return horizontal_walls, vertical_walls, gruben

8
9     def create_matrix_for_laby(horizontal_walls, vertical_walls, gruben, width, height):
10         cost_obj = Cost(horizontal_walls, vertical_walls, gruben, width, height)
11         return cost_obj.create_cost_matrix()
12
13
14     def moving(cost_matrix_1, cost_matrix_2, width, height, horizontal_walls_1, vertical_walls_1, horizontal_walls_2,
15               vertical_walls_2, gruben_1, gruben_2, l):
16
17
18         solving_obj = Solving(cost_matrix_1, cost_matrix_2, vertical_walls_1, horizontal_walls_1, vertical_walls_2,
19                               horizontal_walls_2, gruben_1, gruben_2, width, height)
20
21         moves = [Move([], ((0, 0), (0, 0)), 0, heights_cost=0, weight=0)]
22         while True:
23
24             # Erstes Element aus der Liste nehmen
25             move = moves[0]
26             moves.pop(0)
27
28             # Liste mit den neuen Wegen erweitern
29             new_moves = solving_obj.neighbours_cost(move)
30             moves.extend(new_moves)
31
32             moves = sorted(moves, key=lambda obj: (-obj.weight, len(obj.moves)))
33             moves = moves[:l] # Nur die l besten Züge speichern
34
35             if moves[0].cost == 0:
36                 return moves[0]
37
38
39     def output(move):
40         print(len(move.moves))
41         print(move.moves)
42
43
44     def main():
45         # Datei Einlesen
46         width, height, data = read_data_from_file('input/labyrinth8.txt')
47
48         # Listen erstellen
49         horizontal_walls_1, vertical_walls_1, gruben_1 = create_matrix(data, height)
50         horizontal_walls_2, vertical_walls_2, gruben_2 = create_matrix(data, height)

```

```
52  # Matrix für beide Labyrinth erstellen
    cost_matrix_1 = create_matrix_for_laby(horizontal_walls_1, vertical_walls_1, gruben_1, width, height)
54  cost_matrix_2 = create_matrix_for_laby(horizontal_walls_2, vertical_walls_2, gruben_2, width, height)

56  # Weg finden

58  # TODO: Wert für l anpassen
    l = 100
60  try:
        move = moving(cost_matrix_1, cost_matrix_2, width, height, horizontal_walls_1, vertical_walls_1,
62                    horizontal_walls_2, vertical_walls_2, gruben_1, gruben_2, l)
        output(move)

64  except:
66      print("No solution found")

68
    if __name__ == "__main__":
70        main()
```